

Depth without friction

A live F1 companion, informed by Linear and Apple

Research and product implications | September 7, 2026

Prepared for the founder and future design and engineering team of the F1 app for iOS and macOS.

Build the experience around understanding the live race. Make its depth easy to reach, its state trustworthy, and its controls feel natural on each device.

Live race coverage is the app's main experience. The F1 schedule and race replays support it. The aim is a companion that offers detailed race information while keeping navigation clear and controls responsive. Linear and Apple provide references for that work. The f1-race-replay project remains a possible backend reference, with no authority over the product's interface or feature hierarchy.

Linear's March 2026 refresh preserves information density while reducing competition from navigation. Apple's WWDC26 design principles explain that simplicity comes from reducing effort and clarifying meaning; added context can make an experience easier to understand. Neither requires an empty interface. [\(2026 interface refresh\)](#) [\(Apple design principles\)](#)

A fan should quickly understand what is happening, inspect why it matters, and return to the live session without losing their place. The display must also communicate freshness: stale race information can mislead even when it is presented clearly.

This report examines product philosophy, interaction details, engineering practices, platform differences, accessibility, and relevant sports features. Its final pages propose an F1 experience and a way to evaluate it. Those proposals are design recommendations, not approved specifications or claims that a data provider already supports them.

Live coverage may ultimately accompany a broadcast or include video. That decision remains open. This research does not assess broadcast rights, commercial data access, provider latency, or the replay reference's implementation.

How to read the evidence

Linear sources describe its own practices and releases. Apple sources primarily prescribe design behavior or announce capabilities. Published interaction descriptions were examined; neither company's full product was independently audited. Source notes are linked on each page. Research spans historical examples and material available by September 7, 2026.

Source notes

Linear, Charlie Aufmann and Maxime Heckel. [A calmer interface for a product in motion](#). 2026-03-12.

Apple Developer, Apple design evangelists Linda and Doug. [Principles of great design](#). WWDC26 (2026).

Linear: start with a clear product purpose

Let the product do the organizing

Linear Method favors purpose-built workflows, familiar language, and low setup effort. It describes a product that is approachable initially and reveals more capability as needs grow. This is a useful explanation for how opinionated software can feel powerful without asking every user to become its administrator. ([Linear Method](#))

For this app, the central job should be to help a fan follow and understand the current session. The first useful screen should work before they choose columns, build a layout, or set favorites. Personalization can improve the experience after it already has value.

Find the concept before refining the surface

In "Design is more than code," Karri Saarinen distinguishes understanding a problem, choosing a conceptual form, and testing it through implementation. His projects example shows how a product's purpose can justify a distinct entity instead of whichever abstraction is easiest to code. He also acknowledges that consumer products can require faster testing of uncertain motivations. ([Design is more than code](#))

The F1 implication is to reason about a session, driver, incident, and moment before choosing visual components. A session can connect the schedule, live coverage, results, and replay. That is a proposed product model, not a conclusion derived from the replay repository.

Keep useful preferences

Linear's settings essay distinguishes decisions the product should make well from preferences that legitimately differ between people. It presents settings as a place to understand capabilities, not merely a dumping ground for unresolved decisions. ([Settings essay](#))

Favorites, units, notification interests, and display comfort are plausible preferences here. The app should decide its default hierarchy. A fan should not need to configure whether a race is live or whether important context is visible.

Design for repeated use

Our interpretation: the desired feel comes from accumulating small reductions in effort across an entire race weekend. Consistent naming, predictable navigation, preserved context, and clear defaults matter as much as the first screenshot. Assess a complete journey from an upcoming practice session to live coverage to later review.

Source notes

Linear Method. [Principles & Practices](#). Undated. Accessed September 7, 2026.

Linear, Karri Saarinen. [Design is more than code](#). 2025-12-19.

Linear, Adrien Griveau. [Settings are not a design failure](#). 2022-02-02.

Linear: make quality a working practice

Keep ownership through the whole experience

Saarinen's quality essay describes small teams, continued involvement through delivery, and internal testing of unfinished work. His remote-work account adds concise context, feature flags, selected customer access, and recurring critiques. Team-size descriptions vary between sources; the defensible principle is focused ownership, not a fixed staffing formula. ([Quality essay](#)) ([Remote-work practices](#))

Linear's design guidance separates problem verification, concept exploration, and feedback on details. This helps avoid polishing a concept that has not yet solved the right problem. ([Design project guidance](#))

For the F1 app, review two different things: whether a viewer can understand a race situation, and whether the controls behave consistently. A beautiful driver panel does not establish that its information answers the viewer's question.

Train attention through recurring small fixes

Quality Wednesdays asks engineers to find and correct small experiential imperfections. Linear reports more than 1,000 improvements in two years. Documented examples include adjacent buttons with inconsistent dimensions, a composer whose height shifted when adding a line, and inconsistent hover-exit animation. The exercises combine individual observation with group review. ([Quality Wednesdays](#))

For the F1 app, regularly inspect typography, layout stability, input, accessibility, and interruption in the running experience. Save concrete observations, then fix and demonstrate them.

Treat defects as decisions with owners

Linear's detailed zero-bugs policy sets 48 hours for high-priority bugs and seven days for others, while allowing an explicit decision not to fix a marginal issue. The defining constraint is that bugs cannot be deferred into an indefinite backlog. This is a triage policy, not evidence that Linear has no defects. It qualifies the simpler seven-day wording in Saarinen's May 2025 quality essay. ([Quality essay](#)) ([Zero-bugs policy](#))

For a live service, response urgency must follow the impact on an active race. A wrong live state and a minor spacing inconsistency require different handling. Preserve the accountability; determine operational response targets from the actual service.

Source notes

Linear, Karri Saarinen. [Why is quality so rare?](#). 2025-05-27.

Linear, Karri Saarinen. [Designing remote work at Linear](#). 2025-10-29.

Linear Method. [Manage design projects](#). Undated. Accessed September 7, 2026.

Linear, Tuomas Artman. [Quality Wednesdays: How we trained our team to see what doesn't work](#). 2025-08-13.

Linear, Tuomas Artman and Sabin Roman. [Why we committed to a zero-bugs policy](#). 2025-09-24.

Linear: engineering makes the interface dependable

Separate responsiveness from authority

Linear's August 2026 sync article describes local databases that let local actions and navigation avoid network round trips. Reconnecting clients request changes after a checkpoint. Its revised read path preserves an authoritative source for recent changes, deduplicates overlapping ranges, supports fallback, and was compared against the old path before rollout. These are reported implementation mechanisms, not independently measured results. ([Delta sync engineering](#))

The relevant F1 lesson is architectural: keep inspection and navigation responsive while race facts remain tied to their source. A cache can preserve the last known state during an outage. It cannot make that state current. Timestamps, event identity, ordering, corrections, and reconnect behavior belong in the product model.

Encode the details that must stay consistent

Linear's August 2026 StyleX migration account describes external restyling, broad component APIs, and a long tail of redesign regressions. It explicitly says Linear lacked a formal design system. The remedy included explicit styling contracts, predictable style resolution, shared primitives, and enforcement tooling. ([StyleX engineering](#))

The reported regressions show that attention to design can coexist with fragile implementation patterns. For our app, recurring controls should share their states and behavior so they work consistently together. The article does not select React or StyleX for an Apple-platform product.

Shorten the feedback loop inside the real interface

The 2026 refresh used an in-app toolbar to compare old and new behavior under feature flags. A color tool let the team adjust tokens against the actual UI and transfer values back to design files. The team iterated toward quieter navigation and more consistent action placement while preserving density. ([2026 interface refresh](#))

A development view for this app should be able to replay a stressful race sequence, change text size and appearance, and simulate delayed or missing updates. Those conditions expose focus loss, stale data, or an animation that blocks the next action, which screenshots alone cannot reveal.

Source notes

Linear, Peter Travers. [Rebuilding Linear's delta sync read path](#). 2026-08-18.

Linear, Kenneth Skovhus. [Styling Linear for the future with StyleX](#). 2026-08-26.

Linear, Charlie Aufmann and Maxime Heckel. [A calmer interface for a product in motion](#). 2026-03-12.

Linear's detail work: protect the user's intention

The following are documented examples, selected for transfer to a live race companion. They are not an exhaustive interaction inventory or proof of present behavior in every client.

Make expert actions discoverable

The August 2020 contextual-menu release brought actions to lists, boards, notifications, and related objects, while exposing useful shortcuts. ([2020 menu release](#))

Give driver rows clear contextual actions such as follow, compare, and inspect. Show keyboard equivalents on Mac. The main experience should remain usable through visible controls and touch.

Keep menus stable while decisions are being made

January 2023 notes describe delaying a selected-label menu's reordering until it is reopened. They also describe toggling multiple selections without closing the menu. ([January 2023 release](#))

A driver picker should not move its targets as standings change. Preserve focus by driver identity. Live ranking updates can continue elsewhere while the person's selection context stays stable.

Preserve the meaning of a gesture

December 2022 fixes stop a dialog from closing when a press starts inside it and ends on the backdrop. Escape clears selected text before navigating back. ([December 2022 release](#))

Dragging across a chart or selecting text in an incident should not accidentally dismiss its panel. Escape should unwind the current interaction predictably before leaving the session.

Make undo match the action

April 2024 editor work narrows undo after a paste to the last paste and preserves formatting selection state across toolbar visibility changes. ([April 2024 editor release](#))

Follow and comparison choices need clear reversible behavior. Closing and reopening an inspector should retain the meaningful context that the user expects. State preservation should be deliberate rather than an accidental side effect.

Source notes

Linear Changelog. [Better menus and view options](#). 2020-08-26.

Linear Changelog. [Collapsible Sidebar](#). 2023-01-12.

Linear Changelog. [Rewind 2022](#). 2022-12-08.

Linear Changelog. [Editor improvements](#). 2024-04-24.

Linear's detail work: remove uncertainty and visual noise

Give consistent feedback at the right moment

Quality Wednesdays identifies a hover-exit inconsistency: one button darkened immediately while neighboring buttons faded over 150 milliseconds. June 2026 release notes separately describe instant sidebar hover-state changes and less CPU-intensive loading animations. ([Quality Wednesdays](#)) ([June 2026 release](#))

A pointer or touch should receive prompt acknowledgment. Related controls should behave alike. The 150-millisecond example is not a universal timing token; entering, exiting, pressing, and moving content have different purposes.

Distinguish absence from failure

June 2026 notes replace a false login-success message with a delivery error. July 2026 notes replace an empty board with an error when too many issues match. ([June 2026 release](#)) ([July 2026 release](#))

"No incidents," "incidents unavailable," and "last updated 40 seconds ago" must mean different things. Preserve useful last-known data and explain the reason it cannot be refreshed. Never display a plausible empty state merely because retrieval failed.

Let the user's reading position survive updates

September 2026 notes fix a project-overview scroll indicator jumping to an earlier section. The same release improves accessible display controls and makes creation commands depend on the current view. ([September 2026 release](#))

An arriving event should not steal scroll position or keyboard focus. When a fan reads older incidents, show that new events exist and provide a return control. When they explicitly follow the newest events, keep that mode clear.

Preserve state through navigation

July 2026 search behavior clears the query on Escape while preserving results, then dismisses search when the query is already empty. ([July 2026 release](#))

Driver search, incident filters, and comparison state should have understandable exit behavior. Avoid one global reset that discards several independent choices.

Together, these examples show how an interface can acknowledge input, protect context, communicate state accurately, and stay predictable as conditions change.

Source notes

Linear, Tuomas Artman. [Quality Wednesdays: How we trained our team to see what doesn't work](#). 2025-08-13.

Linear Changelog. [Team documents](#). 2026-06-04.

Linear Changelog. [Coding sessions on mobile](#). 2026-07-30.

Linear Changelog. [Priority inbox](#). 2026-09-03.

Apple: clarity, agency, and platform coherence

Make the purpose legible

WWDC26's design principles place purpose, agency, familiarity, flexibility, simplicity, craft, and delight in a single account of design. Apple explains that an interface can become simpler by adding the context needed for a decision. It also connects maintenance and responsive behavior with trust. ([Apple design principles](#))

For a live race screen, lap, session status, selected driver, and data freshness are useful context. Hiding them to make the screen sparse can increase the effort required to understand it.

Put original design into the race experience

Apple's WWDC26 branding session distinguishes navigation and global controls from the content layer, where products have more room to express their identity. It recommends adapting brand expression to context and using familiar system behavior. ([Brand identity on iOS](#))

For this app, invest custom design in the timing table, track view, event presentation, and driver comparison. Use familiar menus, settings, selection, and window behavior to keep those race-specific views easy to reach.

Share meaning across devices

The WWDC25 design-system session describes a shared component anatomy that adapts between narrow iPhone layouts and expansive Mac layouts. It supports compact controls in dense Mac inspectors, meaningful grouping, and text labels when a symbol is ambiguous. ([Apple design system](#))

Our proposal is a consistent session, driver, and event model across both platforms, expressed differently. iPhone should support a focused task with accessible touch controls. Mac should support sustained observation, comparison, keyboard use, resizing, and persistent inspection.

Let materials evolve with the system

WWDC26 announces further Liquid Glass refinements for readability, separation, personalization, and accessibility. This demonstrates continued adjustment, not a final visual recipe. The shipping status of every announced OS feature was outside this research. ([WWDC26 system updates](#))

Preserve readable content and use maintained system materials where they fit. Evaluate translucent controls over the busiest timing and track content. Check geometry and API availability when defining supported OS versions.

Source notes

Apple Developer, Apple design evangelists Linda and Doug. [Principles of great design](#). WWDC26 (2026).

Apple Developer, Apple design evangelist Sarah. [Communicate your brand identity on iOS](#). WWDC26 (2026).

Apple Developer, Maria, Apple Design Team. [Get to know the new design system](#). WWDC25 (2025).

Apple Developer, Apple; design segment by Cindy. [Platforms State of the Union \(ASL\)](#). WWDC26 (2026).

Apple: design the complete interaction

Fluidity includes changing your mind

In "Designing Fluid Interfaces," Apple's iPhone X team describes minimizing latency, preserving spatial continuity, and allowing gestures to be interrupted or redirected. Motion should respond to the person's input rather than force them to wait for a fixed sequence. ([Fluid interfaces](#))

For our app, opening a driver should not prevent selecting another driver immediately. A dismissing sheet should respond if the gesture reverses. A replay scrubber should acknowledge movement before slower detail finishes loading. Return to live must remain accessible during these transitions.

Before, during, and after input all matter

"The Life of a Button" examines perceived affordance, labels, press feedback, cancellation, and resulting state. Its example preserves cancellation when a finger leaves a target and renewed activation when it returns. The point is the complete behavior of a control, not an ornate animation after it succeeds. ([The Life of a Button](#))

For "Follow driver," define how the action is discovered, how hover and focus work on Mac, how a press is acknowledged on iPhone, what happens when the gesture is canceled, and how the followed state appears afterward. The result should be understandable without a transient message.

Motion should explain a relationship

The WWDC25 design-system session illustrates controls and presentations connected to their point of origin. That spatial relationship helps explain what caused a change. ([Apple design system](#))

An inspector in this app should feel attached to the selected driver or event. Animate an actual position change in a way that preserves driver identity. Ordinary number refreshes need not move the whole row. With reduced motion enabled, keep the information and continuity through a quieter transition.

Make sensory feedback selective

Apple's audio-haptic guidance emphasizes clear cause and consistent meaning across visual, tactile, and audio feedback. Repeated or excessive effects can overwhelm. ([Audio-haptic design](#))

Our proposal is restrained feedback for direct actions and important opted-in moments. A continuously changing timing gap should not continuously vibrate the phone. Evaluate the experience on hardware over a full session, including when sound or haptics are unavailable.

Source notes

Apple Developer, Chan, Nathan de Vries, and Marcos. [Designing Fluid Interfaces](#). WWDC18 (2018).

Apple Developer, Julian and Hugo. [The Life of a Button](#). WWDC18 (2018).

Apple Developer, Maria, Apple Design Team. [Get to know the new design system](#). WWDC25 (2025).

Apple Developer. [Practice audio haptic design](#). WWDC21 (2021).

Apple: accessibility and performance shape the design

Give dense information more than one expression

Apple's visual-accessibility session recommends early accommodations, text or shapes alongside color, adaptable contrast, large-text reflow, and support for motion and transparency preferences. Its examples include switching a horizontal layout to a vertical one as text grows. ([Visual accessibility](#))

For F1, team colors, tire compounds, flags, and sector status need labels or symbols with clear meaning. A timing row at large text sizes should change its arrangement or reveal secondary fields on demand. Shrinking text until every column fits defeats the purpose.

Make screen-reader detail controllable

Apple's data-rich VoiceOver session separates a concise primary label from supplementary information available as custom content. This is an accessibility form of progressive disclosure. ([Data-rich VoiceOver](#))

Our proposed timing row announces driver and the most important current state first, with other metrics available on demand. A fan should be able to inspect a selected driver without the entire field repeatedly interrupting them. The correct live announcement cadence still needs testing with VoiceOver users; the source does not prescribe one for motorsport.

Preserve identity while values change

"Demystify SwiftUI performance" explains how dependencies, cheap view updates, and efficient stable identities affect lists, tables, animation, and view lifetime. It recommends measuring a symptom, finding the cause, changing the implementation, and measuring again. ([SwiftUI performance](#))

Key a driver by stable identity, not current position or a changing timing value. Update the smallest useful part of the interface and keep expensive work out of its immediate input path. This is a design constraint for any stack; SwiftUI remains an implementation candidate.

Test the difficult combinations

Proposed checks should combine live update bursts with scrolling, keyboard selection, larger text, reduced motion, light and dark appearances, and a reconnect. Testing each in isolation can miss the moment when several valid behaviors conflict. Numerical latency and resource budgets should be set after the target devices and feed behavior are known.

Source notes

Apple Developer, Drew Haas. [Make your app visually accessible](#). WWDC20 (2020).

Apple Developer. [Tailor the VoiceOver experience in your data-rich apps](#). WWDC21 (2021).

Apple Developer. [Demystify SwiftUI performance](#). WWDC23 (2023).

Apple Sports: depth across several levels of attention

Let the relevant event lead

Apple's May 2026 Sports release describes a fast, personalized experience organized around followed teams and competitions. It also adds tournament brackets and starting formations. This is a concrete example of adding analytical context to a simple entry point. It does not establish which F1 data is available to another app.

(Apple Sports, May 2026)

For our product, the current or next session should lead. Favorite drivers can shape what is emphasized while the overall race remains understandable. Schedule browsing and replay should connect back to that same race-weekend structure.

Extend the app without duplicating its whole interface

The September 2025 Sports release adds widgets across iPhone, iPad, and Mac, and scheduled Live Activities for upcoming events. This supports a distinction between opening the app to explore and glancing at a smaller surface to stay informed. A Mac widget is not evidence of a full native Mac Sports application. (Apple Sports, September 2025)

Our proposed small surface would show session progress and selected driver context, then open the relevant live session. It should not attempt to compress the full timing table into the Dynamic Island.

Spend interruptions carefully

The Live Activities HIG recommends concise information for an ongoing event, updates when content changes, and alerts only for essential changes. It advises against also sending a push notification for the same update.

(Live Activities guidance)

A race companion should distinguish ordinary data refreshes from events the person chose to hear about. Relevance and duplication matter more than maximizing the count of alerts.

Make stale status explicit

UIKit's push guidance describes a stale-date mechanism and system notification budgets. Background delivery should not be treated as an unlimited, guaranteed-frequency telemetry channel. (UIKit push guidance)

Design distinct fresh, delayed, stale, and finished states for this app's glanceable experience. If a Live Activity cannot update, it should stop implying that its numbers are current. Detailed freshness thresholds depend on the eventual feed and system behavior.

The two developer documentation claims above are supported by visible indexed text from official pages. Direct page extraction returned JavaScript shells; implementation details should be rechecked against the supported SDK.

Source notes

Apple Newsroom. [Apple Sports expands to more than 90 new countries and regions](#). 2026-05-19.

Apple Newsroom. [Apple Sports adds widgets and expands to seven new countries](#). 2025-09-16.

Apple Human Interface Guidelines. [Live Activities](#). Undated living documentation. Accessed September 7, 2026. Official indexed excerpt.

Apple Developer Documentation. [Starting and updating Live Activities with UIKit push notifications](#). Undated living documentation. Accessed September 7, 2026. Official indexed excerpt.

What transfers, and where judgment is required

The following comparison is our synthesis of the preceding evidence. It translates principles into product decisions; it is not a description of an existing F1 app.

Tension	Proposed decision for the F1 companion
Depth and visual calm	Keep core timing visible. Organize detail around a selected driver, event, or comparison. Use alignment and hierarchy to reduce competing emphasis.
Familiar controls and distinctive identity	Preserve platform navigation and input conventions. Create distinctive race-specific content and visual explanations.
Fast feedback and truthful live state	Acknowledge local actions promptly. Show race facts only with their known state and freshness. A loading animation cannot stand in for evidence.
Updating ranks and stable reading	Preserve focus by driver identity. Keep a selected driver's inspector stable. Make event-feed following an explicit mode.
Strong defaults and personal control	Provide an immediately useful live view. Add meaningful preferences such as favorites, alert interests, and comfort settings.
Fluid motion and continuous attention	Animate changes that explain identity or navigation. Keep ordinary updates quiet and provide reduced-motion behavior.
Cross-device consistency and different tasks	Reuse the session model and semantics. Let Mac support concurrent inspection and let iPhone emphasize a focused flow.
Shared components and visual exceptions	Define common states and tokens in code. Permit exceptions when the race experience requires them and review their behavior deliberately.

Avoid overreading the references

Linear's lack of a formal design system in its 2026 engineering account is compatible with having shared components and design tokens. Its zero-bugs policy is compatible with explicit decisions not to fix marginal bugs. Its preference for small teams does not establish an optimal headcount for this project. [\(StyleX engineering\)](#) [\(Zero-bugs policy\)](#)

Apple's developer guidance describes desired behavior. It does not reveal all of Apple's internal operating practices, nor prove every Apple product meets that standard. Press releases establish announced features and positioning, not independent usability or latency results.

The research justifies borrowing specific methods and standards. It does not justify claiming that visual resemblance will produce the same quality.

Source notes

Linear, Kenneth Skovhus. [Styling Linear for the future with StyleX](#). 2026-08-26.
Linear, Tuomas Artman and Sabin Roman. [Why we committed to a zero-bugs policy](#). 2025-09-24.

Proposed experience: the live session is the home of the product

Everything on this page is a recommendation for exploration. Feature availability, names, defaults, and scope remain to be decided.

Organize around the race weekend

Use a session as the link between upcoming schedule, live coverage, final results, and replay. During a session, make returning to live easy from any driver or incident detail. Between sessions, make the next scheduled event and relevant recent sessions easy to find. The interface should explain its current state instead of becoming an empty live dashboard.

Give the live view a clear reading order

First establish the session, lap or time context, race-control state, and freshness. Then make the running order and meaningful gaps easy to scan. Let a selected driver reveal supporting context such as tire state, recent pace, or relevant events where the source supports those fields. Keep interpretation distinguishable from reported facts.

The track view earns prominent space if it helps explain a battle, location, or incident. It should be evaluated against those tasks rather than included as a large decorative centerpiece by default.

Adapt depth to the device

On Mac, explore a central timing view with a persistent driver or event inspector, resizable regions, and keyboard navigation. On iPhone, explore a legible running order with a focused detail presentation and a dependable return path. Share selection semantics and names; let available space and input determine the arrangement.

Connect replay to the same understanding

Replays should reuse the session's concepts while making historical position in time unmistakable. If a viewer scrubs backward during live coverage, show how far behind they are and retain an obvious "Back to live" control. Spoiler preferences and synchronization with a separate broadcast are promising areas to test, not assumed requirements.

Help a fan answer real questions

Use questions such as "Who is this driver racing?", "What changed since I looked away?", and "Why is this interval different?" to evaluate the view. Supply explanations only when the available evidence supports them. Unverified causal claims about strategy or incidents should not be presented as fact.

These choices apply the purpose, hierarchy, control, and contextual-depth findings from Linear and Apple. ([Linear Method](#)) ([Apple design principles](#))

Source notes

Linear Method. [Principles & Practices](#). Undated. Accessed September 7, 2026.

Apple Developer, Apple design evangelists Linda and Doug. [Principles of great design](#). WWDC26 (2026).

Proposed interaction contracts for the F1 app

These are acceptance criteria for later prototypes. None has been implemented or tested yet.

Situation	Expected behavior
A timing value changes	Keep column geometry stable. Use tabular numerals where helpful. Make meaningful changes legible without flashing every refresh.
Two drivers exchange position	Preserve driver identity and selection. Show the new order clearly. Respect reduced motion and never transfer focus to a different driver by index.
The fan opens a driver	Acknowledge input promptly. Keep session context accessible. Accept another selection during the transition.
An event arrives while reading history	Preserve the reading position. Show that newer events are available. Resume automatic following only when the fan chooses it.
The fan scrubs into the past	Label the historical time or lap. Keep the live edge visible. Do not let a live update silently replace the inspected moment.
The feed stops arriving	Retain last-known data with its age. Distinguish session pause from connectivity or provider failure when the evidence allows.
The connection returns	Reconcile updates and corrections without duplicate incidents or unexpected selection changes. Explain any unfillable gap.
A field is unavailable	Show an explicit unavailable state or explanation. Do not substitute zero, empty success, or an inferred measurement.
An alert is relevant	Honor the person's choices. Avoid duplicate notification channels for the same event. Keep ordinary updates silent.
A layout becomes narrow or text grows	Reflow or disclose secondary fields. Preserve essential labels, readable content, reachable actions, and focus.

Review every control through its lifecycle

For important controls, specify rest, hover or focus, press, cancellation, active state, loading, failure, and recovery wherever they apply. Assess transitions between states as carefully as the states themselves. Native controls should retain their standard behaviors unless a specific product need requires a change.

These criteria translate documented continuity, complete-feedback, accessibility, and freshness principles into this product's domain. ([Fluid interfaces](#)) ([The Life of a Button](#)) ([Visual accessibility](#)) ([UIKit push guidance](#))

Source notes

Apple Developer, Chan, Nathan de Vries, and Marcos. [Designing Fluid Interfaces](#). WWDC18 (2018).

Apple Developer, Julian and Hugo. [The Life of a Button](#). WWDC18 (2018).

Apple Developer, Drew Haas. [Make your app visually accessible](#). WWDC20 (2020).

Apple Developer Documentation. [Starting and updating Live Activities with UIKit push notifications](#). Undated living documentation. Accessed September 7, 2026. Official indexed excerpt.

Proposed build approach and remaining questions

Build a small, complete live experience first

Start with a representative session and the core path: open live, scan the order, inspect a driver, read a change, and return. Compare alternative hierarchies before polishing one. Recorded data can help rehearse behavior, but a real-feed feasibility check must happen early enough to shape the product. The replay example can inform that investigation without becoming the product blueprint.

Keep a focused set of shared components for rows, metrics, status, selection, panels, and controls. Define their meanings and states alongside visual tokens. Add realistic state fixtures for an ordinary session, rapid changes, interruption, missing fields, corrections, and reconnect. These recommendations apply Linear's collaborative iteration and component lessons to this workload. ([Design project guidance](#)) ([StyleX engineering](#))

Make quality observable

Track whether fans correctly understand race situations, whether interactions keep their place, and whether live state is truthful. Measure input response, update age, lost or duplicate events, scrolling under load, and resource use on the intended devices. Keep acquisition latency separate from rendering latency. Set targets from actual hardware and feed evidence instead of borrowing a headline number from another app.

Use a recurring quality review with small demonstrable fixes, paired with a separate response process for correctness and service failures. Recheck performance after a change. These practices draw from Quality Wednesdays and Apple's measurement loop. ([Quality Wednesdays](#)) ([SwiftUI performance](#))

Resolve the consequential unknowns next

The next investigation should establish what live coverage includes, especially whether video is part of it; available data and permitted uses; update latency and reliability; supported devices and OS versions; and which depth fans most value during a session. Broadcast delay alignment, spoilers, and the treatment of inferred strategy should be explored with viewers.

Evidence limits and stopping point

This is a bounded primary-source study, not an exhaustive catalog of every interaction. Historical examples were checked against relevant 2026 material. Two Apple documentation pages were usable through indexed excerpts only. Announced WWDC26 changes are not asserted to be universally shipped. No live product benchmark, hardware interaction test, feed assessment, or repository audit was performed.

Research stopped once the major philosophy, interaction, engineering, accessibility, and platform claims had primary support and the consequential differences were explained. The sources support this design direction. The exact feature set and implementation still require product and technical evidence.

Source notes

Linear Method. [Manage design projects](#). Undated. Accessed September 7, 2026.

Linear, Kenneth Skovhus. [Styling Linear for the future with StyleX](#). 2026-08-26.

Linear, Tuomas Artman. [Quality Wednesdays: How we trained our team to see what doesn't work](#). 2025-08-13.

Apple Developer. [Demystify SwiftUI performance](#). WWDC23 (2023).